pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

NumPy, Part 2

WEEK 2, DAY 1, PART 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's ninth note

This morning's answers covered whole columns. These are about part of one: which bonds are below par, which are high yield, and what if the cheap ones rally a point. Then check our 98.50 on QVNE against its cash flows, value the book for Monday to Thursday, and save it all for tomorrow.

Part 1's arrays in one cell

```
import numpy as np
tickers = np.array(['QVNE', 'DNMR', 'PLWK', 'HLVF'])
coupons = np.array([5.25, 6.125, 4.75, 7.5])
prices = np.array([98.5, 103.25, 95.0, 101.0])
faces = np.array([1500000, 500000, 750000, 250000])
scores = np.array([9, 11, 7, 14])
incomes = faces * coupons / 100
current_yields = coupons / prices * 100
yields_with_zero = np.array([current_yields[0], np.inf, current_yields[2], current_yields[3]])
print(yields_with_zero)
[5.32994924        inf 5.         7.42574257]
```

scores is week 1's rating score: AAA is 1, one more per notch lower; 11 or higher is high yield.

6.5 Picking out bonds by position

```
print(prices[0])
print(tickers[2])
print(coupons[0:2])
print(faces[-1])
prices[0]
98.5
PLWK
[5.25  6.125]
250000
np.float64(98.5)
```

Shown bare, 98.5 carries its type, np.float64; print() shows the value alone.

One past the end

```
print(prices[4])
IndexError: index 4 is out of bounds for axis 0 with size 4
```

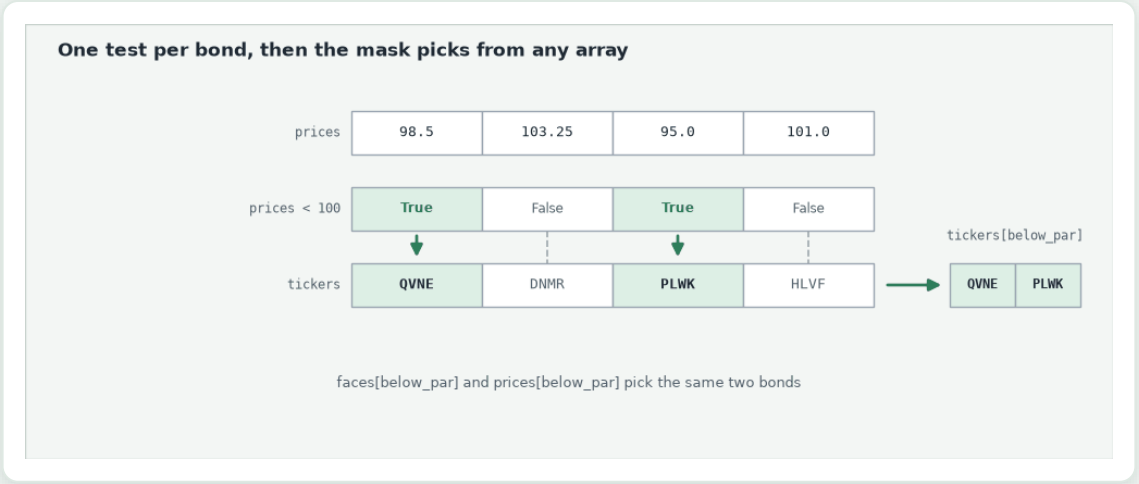
Size 4: positions 0 to 3, the last always the size minus 1.

Several positions at once

```
print(tickers[[0, 2]])
print(prices[np.arange(0, 4, 2)])
['QVNE' 'PLWK']
[98.5 95. ]
```

Below par: a mask picks the bonds

```
below_par = prices < 100
print(below_par)
[ True False  True False]
print(tickers[below_par])
print(prices[below_par])
print('Bonds below par:', below_par.sum())
print('Face below par:', faces[below_par].sum())
['QVNE' 'PLWK']
[98.5 95. ]
Bonds below par: 2
Face below par: 2250000
```



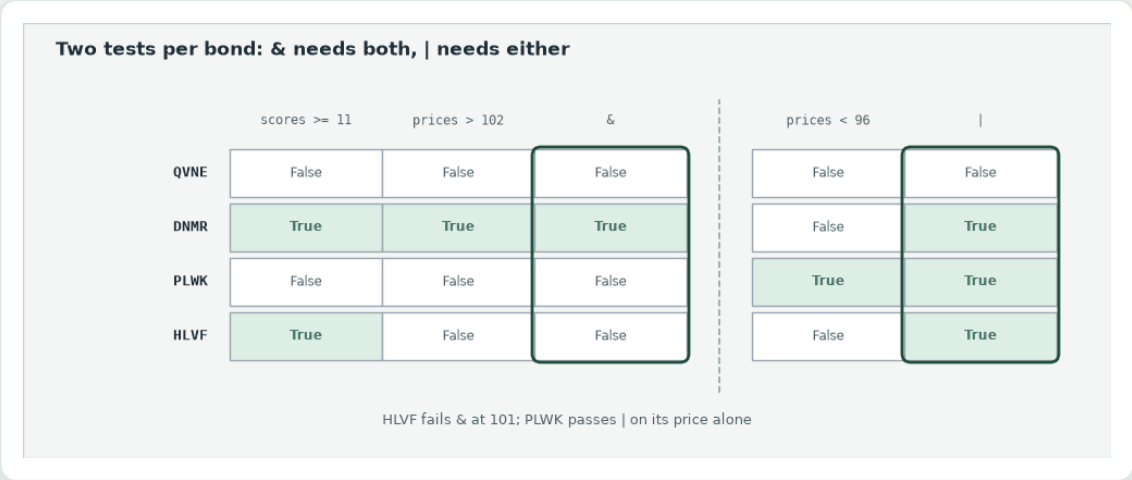
A comparison on an array gives one True or False per bond: a boolean mask.

One comparison, four answers, one bond order.

High yield, both tests or either, then labels

```
print(tickers[scores >= 11], incomes[scores >= 11].sum())
print(tickers[(scores >= 11) & (prices > 102)])
print(tickers[(scores >= 11) | (prices < 96)])
['DNMR' 'HLVF'] 49375.0
['DNMR']
['DNMR' 'PLWK' 'HLVF']
```

& means and, | means or, for masks; brackets make each test run first.



Real yields only

```
real = np.isfinite(yields_with_zero)
print(real)
print(yields_with_zero[real])
[ True False  True  True]
[5.32994924 5.         7.42574257]
```

A label for each bond

```
labels = np.where(prices > 100, 'Premium', 'Discount')
print(labels)
['Discount' 'Premium' 'Discount' 'Premium']
```

No loop and no if, then a what-if on a copy

Excel side by side: ticker in A, face in B, price in C, yield in F

Question	Python	Excel
Keep the real yields	`np.isfinite(yields_with_zero)`	=ISNUMBER(F2) filled down: FALSE for #DIV/0!
Label each bond	`np.where(prices > 100, 'Premium', 'Discount')`	=IF(C2>100, "Premium", "Discount") filled down
Count the bonds below par	`(prices < 100).sum()`	=COUNTIF(C2:C5, "<100")
Face below par	`faces[prices < 100].sum()`	=SUMIF(C2:C5, "<100", B2:B5)
Tickers below par	`tickers[prices < 100]`	=FILTER(A2:A5, C2:C5<100) (Excel 2021, Microsoft 365)

```
what_if = prices.copy()
what_if[below_par] = what_if[below_par] + 1
print('What-if: ', what_if)
print('Original:', prices)
What-if:  [ 99.5  103.25  96.   101.  ]
Original: [ 98.5  103.25  95.   101.  ]
```

```
edited = prices.copy()
edited[0:2] = [99.0, 104.0]
edited[2:] = 100
print(edited)
[ 99. 104. 100. 100.]
```

A mask or a slice on the left of = changes just those bonds. Now the note's bond.

Predict first: decide, then run it in the notebook

Predict 1

A slice of an array is an array. The next cell sets the first value of the slice "prices[0:2]" to 0. What does "prices" print? A) 98.5 first. B) 0 first. C) An error.

```
first_two = prices[0:2]
first_two[0] = 0
print(prices)
```

Predict 2

"and" joined one True or False to another. The next cell puts it between two masks. What happens? A) DNMR, as with &. B) An error. C) An empty array.

```
print(tickers[(scores >= 11) and (prices > 102)])
```

6.6 QVNE's cash flows and discount factors

A bond's price is the present value of its cash flows: each payment times its discount factor (week 1's Part 1), added up.

Assumption	Meaning
Whole periods	7 years exactly: 14 semiannual coupons
Semiannual compounding	half the annual yield per period
Settlement on a coupon date	no accrued interest

Where e comes from

In 1683 Jacob Bernoulli, working on compound interest, looked for the limit of (1 + 1/n) to the power n as n grows, and showed it lies between 2 and 3.

Source: MacTutor History of Mathematics, "The number e"

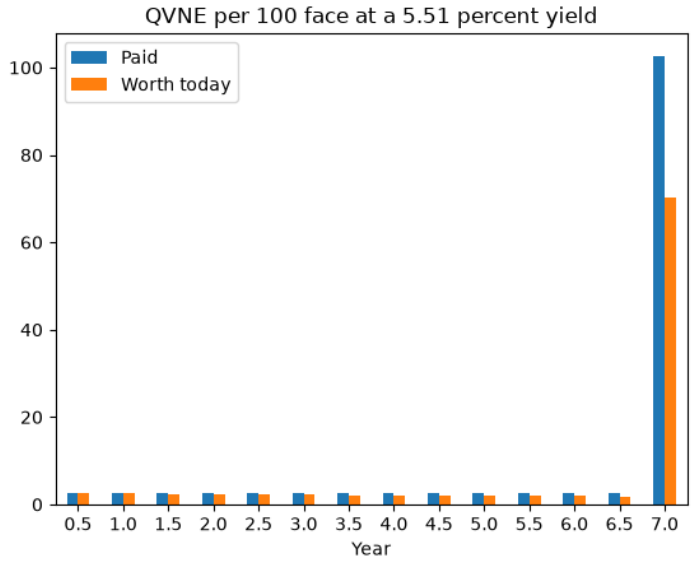
```
periods = np.arange(1, 15)
cash_flows = np.ones(14) * 5.25 / 2
cash_flows[-1] = cash_flows[-1] + 100
print(periods)
print(cash_flows)
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14]
[  2.625   2.625   2.625   2.625   2.625   2.625   2.625   2.625   2.625
   2.625   2.625   2.625   2.625 102.625]
```

```
yield_pct = 5.51
yield_per_period = yield_pct / 100 / 2
discount_factors = 1 / (1 + yield_per_period) ** periods
print(np.round(discount_factors, 4))
[0.9732 0.9471 0.9217 0.897  0.8729 0.8495 0.8268 0.8046 0.783  0.762
 0.7416 0.7217 0.7024 0.6835]
```

QVNE's price, from its cash flows

```
present_values = cash_flows * discount_factors
print(np.round(present_values, 4))
qvne_price = present_values.sum()
print('QVNE at 5.51 percent:', round(qvne_price, 4))
[ 2.5546  2.4861  2.4195  2.3546  2.2915  2.23    2.1702  2.1121  2.0554
  2.0003  1.9467  1.8945  1.8437 70.1474]
QVNE at 5.51 percent: 98.5067
```

```
payment_years = periods / 2
continuous_factors = np.exp(-yield_pct / 100 * payment_years)
continuous_price = (cash_flows * continuous_factors).sum()
print('Twice a year:', round(qvne_price, 4))
print('Continuous:  ', round(continuous_price, 4))
Twice a year: 98.5067
Continuous:   98.0718
```



Excel side by side

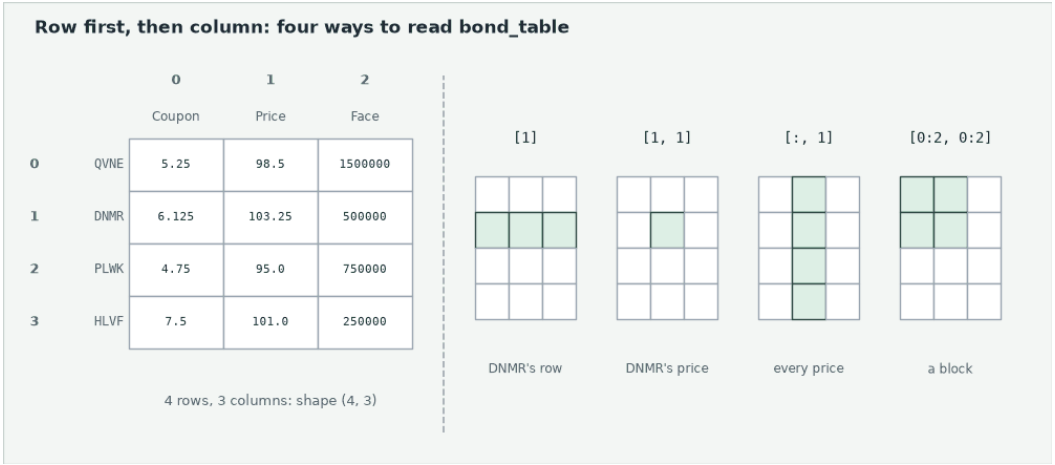
=SUMPRODUCT(B2:B15, C2:C15) on the cash flows and discount factors, or =PV(5.51%/2, 14, -2.625, -100) in one step, gives 98.5067.

98.5067: within a cent of the desk's 98.50, with no loop.

6.7 Four bonds, three columns

```
bond_table = np.array([[5.25, 98.5, 1500000],
                       [6.125, 103.25, 500000],
                       [4.75, 95.0, 750000],
                       [7.5, 101.0, 250000]])
[[      5.25       98.5   1500000.    ]
 [      6.125     103.25   500000.    ]
 [      4.75       95.     750000.    ]
 [      7.5       101.     250000.    ]]
ndim: 2 shape: (4, 3)
```

```
print(bond_table[1])
print(bond_table[1, 1])
[     6.125    103.25  500000.    ]
103.25
print(bond_table[:, 1])
print(bond_table[0:2, 0:2])
[ 98.5  103.25  95.   101.  ]
[[  5.25   98.5  ]
 [  6.125 103.25 ]]
```




```
print(bond_table.mean(axis=0))
[     5.90625     99.4375  750000.     ]
```

Row first, then column; a colon alone takes them all.

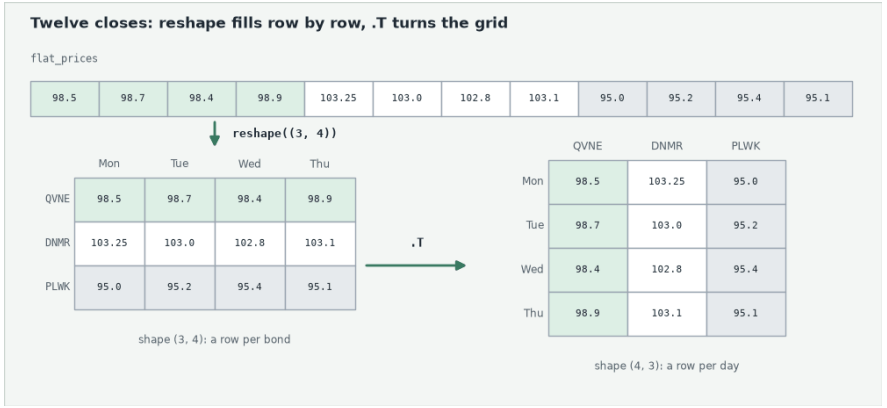
A run of twelve closes: reshape, then turn

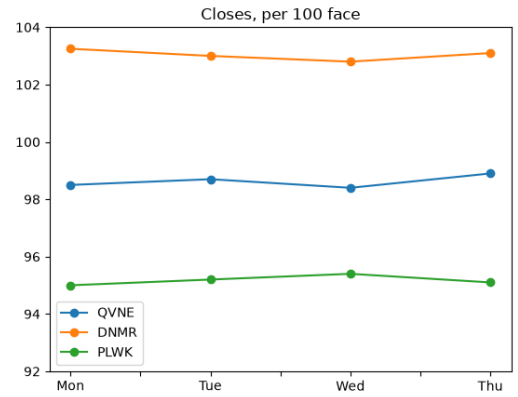
```
flat_prices = np.array([98.5, 98.7, 98.4, 98.9,
                        103.25, 103.0, 102.8, 103.1,
                        95.0, 95.2, 95.4, 95.1])
price_grid = flat_prices.reshape((3, 4))
print(price_grid)
[[ 98.5   98.7   98.4   98.9 ]
 [103.25 103.   102.8  103.1 ]
 [ 95.    95.2   95.4   95.1 ]]
```

```
flat_prices.reshape((3, 5))
ValueError: cannot reshape array of size 12 into shape (3,5)
```

```
print(price_grid.T)
print(price_grid.T.shape)
[[ 98.5  103.25  95.  ]
 [ 98.7  103.    95.2 ]
 [ 98.4  102.8   95.4 ]
 [ 98.9  103.1   95.1 ]]
(4, 3)
```

Excel side by side. In Microsoft 365, with the twelve in A1:L1, =WRAPROWS(A1:L1, 4) spills the same grid.





Predict first: three more

Predict 3

What is QVNE's price at a yield of 5.25 percent, its own coupon? A) Above 100. B) Below 100. C) Exactly 100.

```
def price_from_yield(coupon, years, yield_pct):
    periods = np.arange(1, years * 2 + 1)
    cash_flows = np.ones(periods.size) * coupon / 2
    cash_flows[-1] = cash_flows[-1] + 100
    discount_factors = 1 / (1 + yield_pct / 100 / 2) ** periods
    return (cash_flows * discount_factors).sum()
print('At 5.51:', round(price_from_yield(5.25, 7, 5.51), 4))
print('At 5.25:', round(price_from_yield(5.25, 7, 5.25), 4))
```

Predict 4

"payments" has 4 rows and 2 columns. How many numbers does "payments.sum(axis=1)" give? A) 2 B) 4 C) 1

```
payments = np.array([[39375.0, 39375.0], [15312.5, 15312.5],
                     [17812.5, 17812.5], [9375.0, 9375.0]])
print('axis=1:', payments.sum(axis=1))
print('axis=0:', payments.sum(axis=0))
print('no axis:', payments.sum())
```

Predict 5

"@" is a new symbol. What does "cash_flows @ discount_factors" give? A) 14 numbers, value by value, as * gives. B) One number. C) An error.

```
print(np.round(cash_flows @ discount_factors, 4))
```

6.8 Keep it past today

```
np.save('desk_prices.npy', prices)
loaded_prices = np.load('desk_prices.npy')
print(loaded_prices)
[ 98.5  103.25  95.   101.  ]
```

```
np.savetxt('desk_bond_table.csv', bond_table, delimiter=',', fmt='%.3f',
           header='coupon,price,face', comments='')
with open('desk_bond_table.csv', 'r', encoding='utf-8') as file:
    print(file.read())
coupon,price,face
5.250,98.500,1500000.000
6.125,103.250,500000.000
4.750,95.000,750000.000
7.500,101.000,250000.000
```

```
loaded_table = np.loadtxt('desk_bond_table.csv', delimiter=',',
skiprows=1)
print(np.array_equal(loaded_table, bond_table))
True
```

```
np.savez('desk_arrays.npz', prices=prices, faces=faces)
with np.load('desk_arrays.npz') as saved:
    print(saved.files)
    print(saved['faces'])
['prices', 'faces']
[1500000  500000  750000  250000]
```

File	Holds	Every digit kept	Excel opens it
.npy	one array	yes	no
.npz	several arrays, by name	yes	no
.csv	one table of numbers, as text	only the decimals written	yes

The note, answered

The note asked	Answer
Below par	QVNE and PLWK, 2,250,000 of face, from one mask
High yield	DNMR and HLVF, 49,375 a year
The cheap ones up a point	99.5 and 96, on a copy
QVNE's 98.50	98.5067, from its cash flows
Saved for tomorrow	three files, read back equal

Day 2's pandas puts names on the rows and columns, and the book grows to two hundred bonds.